Basic (your first LCEL chain) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the `Runnable` pattern
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the `Runnable` pattern

This exercise is about the `Runnable` pattern that every LCEL piece shares. A prompt template, a chat model, and an output parser are all `Runnable` objects — each one takes an input and produces an output in a predictable, swappable way.

`ChatPromptTemplate.from_template("Answer: {question}")` builds a template object — it doesn't call anything yet, it just describes the shape of the prompt, with `{question}` as a placeholder. `prompt | ChatOpenAI() | StrOutputParser()` chains three `Runnable`s together using `|`, which LangChain overloads to mean "feed this one's output into the next one's input." The result is itself a single `Runnable`, so `chain.invoke({"question": "..."})` runs all three steps and returns the final parsed answer.

The imports: `ChatPromptTemplate` and `StrOutputParser` live in `langchain_core`, while `ChatOpenAI` lives in `langchain_openai` — this is the package split the Core Concepts section describes. `ChatOpenAI()` reads its API key and default model name from your environment the same way Doc04's raw client did — pass `model=` and `temperature=` explicitly if you want to be sure which model you're using.

Worth knowing beyond `.invoke()`: the exact same chain also supports `.batch(...)` (run it on a *list* of inputs, often concurrently) and `.stream(...)` (get pieces of the output as they're produced) — for free, no extra code. Not needed for this exercise's single-input chain, but useful the moment you're processing more than a handful of inputs.

Hint 2 — The plan, and almost the whole thing

```
import ChatPromptTemplate, ChatOpenAI, StrOutputParser
```

```
prompt = ChatPromptTemplate.from_template("Answer: {question}")

chain = prompt | ChatOpenAI(model="...", temperature=0) | StrOutputParser()

result = chain.invoke({"question": "What's 5 + 7?"})

print(result)
```

```
# lcel_chain_basics_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Answer: {question}")
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
chain = prompt | model | StrOutputParser()
```

Notice the input to `.invoke(...)` is a dictionary whose key (`question`) matches the template's placeholder name exactly. If they don't match, the chain fails while building the prompt, before any API call happens — you'll practice that failure directly in this document's Edge cases exercise. Add the `.invoke(...)` call and print line yourself. Then try `build_chain(config: Config | None = None)`: if `config` is `None`, call Doc01's `load_config()`; pass `api_key=config.openai_api_key` to `ChatOpenAI`. Compare against the Solution.

Full solution: Show me the solution